UMassAmherst | Manning College of Information & Computer Sciences

Programming Methodology

Lab 11: Mock Testing and Program Correctness

Wednesday, April 22nd, 2026

Based on slides by Maximillian Kuechen

Weekly Lab Agenda

- Go over reminders/goals
- Review past material
- Work in groups of 2-3 to solve a few exercises
- Discussion leaders will walk around and answer questions
- Solutions to exercises will be reviewed as a class
- Attendance taken at the end

Reminders

- HW 8 is due next Wednesday (4/29) at 11:59pm
- GET IN TOUCH ASAP with your team for HW8 if you haven't already!
- Make sure you've joined the repo for you team
  - **Almost 50 students still have yet to accept their invites**
  - The original invites have expired, and we've sent out a second round. Please accept the new one and post on Campuswire if you need help!
- Reach out to us on Campuswire privately if there are any issues in your team.
  - Reach out early so we can get you back on track!

Today's Goals

- Practice testing asynchronous code and mock testing
- Practice program correctness

Testing Asynchronous Code

We can use Jest to help us test asynchronous code in various ways.

We will go over some brief examples for an asynchronous function **`MyFunction`**, which takes some sort of input and returns a **`Promise`**.

At the top of our testing file, we can use Jest to define a time limit for running all our tests. The code to the right will set a 30 second time limit (**`jest.setTimeout`** takes milliseconds as input).

```
const SECOND = 1000;
jest.setTimeout(30 * SECOND);
```

Testing Asynchronous Code

Below, **`res`** is a **`Promise`**. We can then use **`expect(res).resolves`** or **`rejects`** to check if the **`Promise`** resolves/rejects, and examine the value that the **`Promise`** resolves/rejects with.

```
describe('Tests for MyFunction', () => {
  it('Testing resolve', () => {
    const res = MyFunction(someInput);
    return expect(res).resolves.toBe(someValue);
  });

  it('Testing reject', () => {
    const res = MyFunction(someInput);
    return expect(res).rejects.toBe(someValue);
  });
});
```

Note that you must include the **`return`** statement, as this allows Jest to keep track of incomplete tests.

Testing Asynchronous Code

Alternatively, we could also use **`.then()`** instead, or **`.catch()`** in a similar fashion. Again, we must include the **`return`** statement.

```
describe('Tests for MyFunction', () => {
  it('Some test', () => {
    const res = MyFunction(someInput);
    return res.then(val =>
      expect(val).toBe(someValue)
    );
  });
});
```

Testing Asynchronous Code

We can also use **`async`**/**`await`**. In order to use **`await`** in a test, we need to specify **`async`** in front of the function passed to the **`it`** block.

```
describe('Tests for MyFunction', () => {
  it('Some test', async () => {
    const res = await MyFunction(someInput);
    expect(res).toBe(someValue);
  });
});
```

When using **`async`**/**`await`**, we do not need the **`return`** statement.

Mock Testing

There are times when we implement a function that calls some other existing function, which may be complex, unpredictable, or expensive to call.

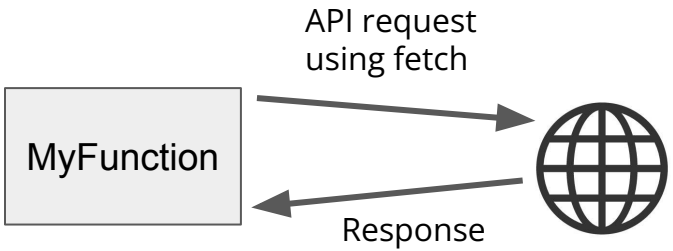
This can make testing difficult, as we may want to avoid calling this other function too much. However, we still want a way to test if our function will work for different outputs of this other function.

We can instead intercept all calls to this function, controlling what is returned. This is known as mock testing.

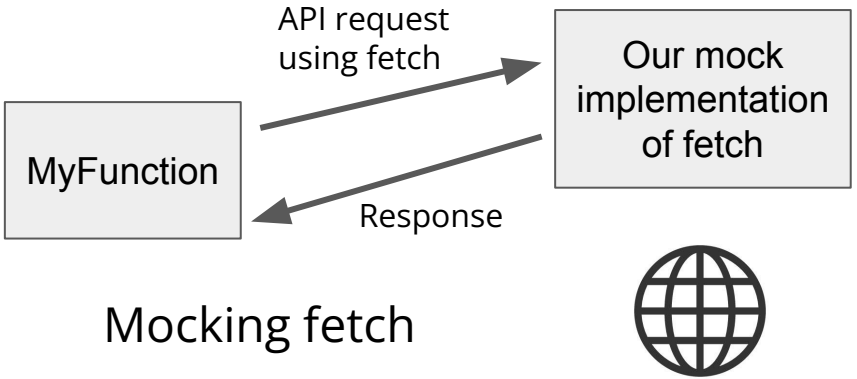
Mocking Fetch

`fetch` is a good example of a function that we can use for mock testing. For instance, if our function relies on an API we can't control, or an API that is expensive to call repeatedly, it would be nice if we could "simulate" the API.

We can do so by mocking **`fetch`**, which allows us to intercept any API calls from our function and return whatever values we want during testing.



Normal API usage



Mocking fetch

Mocking Fetch

We will use the **`jest-fetch-mock`** library to help us mock **`fetch`**.

To mock **`fetch`**, we must first call **`fetchMock.enableMocks()`**. This tells Jest that we intend to mock **`fetch`**.

We can then use **`fetchMock.mockResponse(val)`** to specify what our "intercepted" **`fetch`** should return.

We can use **`JSON.stringify()`** if we want **`fetch`** to return JSON.

```
import fetchMock from "jest-fetch-mock";

describe('Tests for MyFunction', () => {
  it('Some test', () => {
    fetchMock.enableMocks();

    const input = JSON.stringify([1, 2, 3]);
    fetchMock.mockResponse(input);
  });
});
```

Mocking Fetch

```
describe('Tests for MyFunction', () => {
  it('Some test', async () => {
    fetchMock.enableMocks();

    const input = JSON.stringify([1, 2, 3]);
    fetchMock.mockResponse(input);

    const res = await fetch("test_url");
    const arr = await res.json();

    fetchMock.resetMocks();
    fetchMock.disableMocks();
  });
});
```

When we now call **`fetch`**, it will use our implementation. This means **`res`** will be a **`Response`** object. If we call **`.json()`** on this **`Response`** object, we will obtain the array **`[1, 2, 3]`**.

We should then call **`resetMocks()`** to clear our mocks. We can then disable mocks on **`fetch`** with **`disableMocks()`**, which allows **`fetch`** to revert to normal and not be intercepted anymore.

Mocking Fetch

```
describe('Tests that mock fetch', () => {
  beforeEach(() => {
    fetchMock.enableMocks();
  });

  afterEach(() => {
    fetchMock.resetMocks();
    fetchMock.disableMocks();
  });

  // insert tests here
});
```

`enableMocks`, **`resetMocks`**, and **`disableMocks`** need to be in every test that mocks **`fetch`**.

If we have multiple tests that need to mock **`fetch`**, we can reduce code duplication!

We will first define a **`describe`** block for just the tests that need to mock **`fetch`**.

We can then use **`beforeEach`** and **`afterEach`** in this **`describe`** block, which calls the desired functions before and after each test.

Mock Testing

You can learn more about testing asynchronous code and mock testing in the slides linked in the HW 8 specs: Testing Asynchronous Code

To practice mocking fetch, we will introduce a function that we can test by mocking fetch. To help you understand the function, we will ask you to try and implement it for exercise 1.

We will only spend around 5-7 minutes on this exercise; do not worry if you do not solve it in time! Our main focus will be on writing tests for this function.

Exercise 1: Mock Testing

Write tests for the function **`getObjsWithName(url)`** that takes in an array of URLs and returns a **`Promise`** of an array of objects. The array of objects should contain all objects with the property "name" found at every URL.

- You should mock **`fetch`** in all of your tests.
  - Hint: use **`fetchMock.mockImplementation()`** to control what **`fetch`** does depending on the URL it is called on
- You can try running your tests on the flawed solutions in **`lab.js`**.
- **`getObjsWithName`** assumes:
  - All URLs point to valid JSON data.
  - The data is an array of objects.

Exercise 1: Solution

We will write 5 tests:

1. **`getObjsWithName()`** returns an empty array if all promises reject
2. **`getObjsWithName()`** works if all promises fulfill with an array of objects with the name property
3. **`getObjsWithName()`** works if all promises fulfill but not all objects have the name property
4. **`getObjsWithName()`** returns empty array if all promises fulfill but objects have no name field
5. **`getObjsWithName()`** works if mix of promises fulfilling and rejecting

Exercise 1: Solution


```
describe("getObjsWithName", () => {
  beforeEach(() => {
    fetchMock.enableMocks();
  });
  afterEach(() => {
    fetchMock.resetMocks();
    fetchMock.disableMocks();
  });

  it("returns empty array if all promises reject", async () => {
    fetchMock.mockReject(new Error("Test"));
    const urls = ["a", "b", "c"];
    const res = await getObjsWithName(urls);
    expect(res).toHaveLength(0);
  });
  // ...
});
```

We make sure to call **`enableMocks`**, **`resetMocks`**, and **`disableMocks`** before/after all tests, since we are mocking **`fetch`** in all our tests.

We can mock **`fetch`** to always reject to check what happens if all promises reject.

Exercise 1: Solution


```
describe("getObjsWithName", () => {
  it("works if all promises fulfill with an array of objects w/ name property", async () => {
    const objs = {
      a: [{ name: "a1" }, { name: "a2" }],
      b: [{ name: "b1" }, { name: "b2" }],
      c: [{ name: "c1" }, { name: "c2" }],
    };
    fetchMock.mockImplementation(url => {
      const obj = JSON.stringify(objs[url]);
      return Promise.resolve(new Response(obj));
    });
    const urls = ["a", "b", "c"];
    const res = await getObjsWithName(urls);
    expect(res.every(obj => "name" in obj)).toBe(true);
    expect(res.map(obj => obj["name"])).toEqual(["a1", "a2", "b1", "b2", "c1", "c2"]);
  });
  // ...
});
```

We mock **`fetch`** to return arrays of objects with the **`name`** property, using **`mockImplementation`** to return a different response for each URL. We then check that the result is an array of objects with the **`name`** property, and that the names are correct.

Exercise 1: Solution


```
describe("getObjsWithName", () => {
  it("works if all promises fulfill but not all objects have name property", async () => {
    const objs = {
      a: [{ name: "a1" }, { nme: "a2" }, { name: "a3" }],
      b: [{ name: "b1" }],
      c: [{ nam: "c1" }, { n: "c2" }],
    };
    fetchMock.mockImplementation(url => {
      const obj = JSON.stringify(objs[url]);
      return Promise.resolve(new Response(obj));
    });
    const urls = ["a", "b", "c"];
    const res = await getObjsWithName(urls);
    expect(res.every(obj => "name" in obj)).toBe(true);
    expect(res.map(obj => obj["name"])).toEqual(["a1", "a3", "b1"]);
  });
  // ...
});
```

We can test when objects don't all have a **name** property, following a very similar approach as the previous test. This is a great place to reduce code duplication!

Exercise 1: Solution


```
describe("getObjsWithName", () => {
  it("returns [] if all promises fulfill but objects have no name field", async () => {
    const objs = {
      a: [{ game: "a1" }, { fame: "a2" }],
      b: [{}, { tame: "b1", z: "b2" }],
    };
    fetchMock.mockImplementation(url => {
      const obj = JSON.stringify(objs[url]);
      return Promise.resolve(new Response(obj));
    });
    const urls = ["a", "b"];
    const res = await getObjsWithName(urls);
    expect(res).toHaveLength(0);
  });
  // ...
});
```

We can test when no objects have a **name** property, following a very similar approach as the previous tests.

Exercise 1: Solution


```
describe("getObjsWithName", () => {
  it("works if mix of promises fulfilling and rejecting", async () => {
    const objs = {
      a: [{ name: "a1" }, { name: "a2" }],
      c: [{ name: "c1" }, { name: "c2" }],
    };
    fetchMock.mockImplementation(url => {
      if (["b", "d"].includes(url)) {
        return Promise.reject(new Error("Testing rejecting promise"));
      }
      const obj = JSON.stringify(objs[url]);
      return Promise.resolve(new Response(obj));
    });
    const urls = ["a", "b", "c", "d"];
    const res = await getObjsWithName(urls);
    expect(res.every(obj => "name" in obj)).toBe(true);
    expect(res.map(obj => obj["name"])).toEqual(["a1", "a2", "c1", "c2"]);
  });
  // ...
});
```

We can also mock **`fetch`** such that it rejects/resolves depending on the URL given. This allows us to test what happens if there is a mix of resolving/rejecting promises.

What Is an Invariant?

- As its name says, an invariant is something that *does not change*
  - Here: a *specific type of assertion* (not all assertions are invariants)
- We work with **loop invariants**: they hold **before** each loop iteration (before checking the condition)
  - A useful invariant relates variables that *change* within the loop
  - The invariant will not hold at *every* point in the loop but will be *restored* (holds again) at the end

- In OOP, we also work with *method invariants* and *class invariants*

Exercise 2: Program Correctness

What is the largest value **A** for which the loop below terminates with **n**=20?

Use an invariant to justify your answer.

```
let n: number = 0;

let s: number = A;

while (s <= 100) {

     if (n % 2 > 0) {

         s += n;

    } else {

    }

    n = n + 1;

}
```

#Fall 2022 Final Exam

Exercise 2: Solution


```
let n: number = 0;
let s: number = A;
// s = A + oddsum(n)  -  sum of odd numbers less than n
while (s <= 100) {
  // s = A + oddsum(n)
  if (n % 2 > 0) {

    s += n;

  } else {

  }

  n = n + 1;

}
```

The loop adds odd numbers.
Our invariant is that **s** is **A** (initial value) plus the sum of odd numbers less than **n**.

Exercise 2: Solution


```
let n: number = 0;
let s: number = A;
// s = A + oddsum(n)  -  sum of odd numbers less than n
while (s <= 100) {
  // s = A + oddsum(n)
  if (n % 2 > 0) {
    // s = A + oddsum(n)
    s += n;

  } else {

  }

  n = n + 1;

}
```

The loop adds odd numbers.
Our invariant is that **s** is **A** (initial value) plus the sum of odd numbers less than **n**.

Exercise 2: Solution


```
let n: number = 0;
let s: number = A;
// s = A + oddsum(n)  -  sum of odd numbers less than n
while (s <= 100) {
  // s = A + oddsum(n)
  if (n % 2 > 0) {
    // s = A + oddsum(n) && n % 2 > 0
    s += n;
    // s = A + oddsum(n) + n && n % 2 > 0
    // s = A + oddsum(n+1)
  } else {

  }

  n = n + 1;

}
```

We have added **n**, which is odd, so we have the sum of all numbers less than **n**+1.

Exercise 2: Solution


```
let n: number = 0;
let s: number = A;
// s = A + oddsum(n)  -  sum of odd numbers less than n
while (s <= 100) {
  // s = A + oddsum(n)
  if (n % 2 > 0) {
    // s = A + oddsum(n) && n % 2 > 0
    s += n;
    // s = A + oddsum(n) + n && n % 2 > 0
    // s = A + oddsum(n+1)
  } else {
    // s = A + oddsum(n)  && n % 2 === 0
    // s = A + oddsum(n + 1)
  }

  n = n + 1;

}
```

For the **else** block, since **n** is even, **oddsum(n)** = **oddsum(n+1)**.

Exercise 2: Solution


```
let n: number = 0;
let s: number = A;
// s = A + oddsum(n)  –  sum of odd numbers less than n
while (s <= 100) {
  // s = A + oddsum(n)
  if (n % 2 > 0) {
    // s = A + oddsum(n) && n % 2 > 0
    s += n;
    // s = A + oddsum(n+1)
  } else {
    // s = A + oddsum(n+1)
  }
  // s = A + oddsum(n+1)  –  common for both branches
  n = n + 1;

}
```

We have a common form after the **`if`** statement.

Exercise 2: Solution


```
let n: number = 0;
let s: number = A;
// s = A + oddsum(n)  –  sum of odd numbers less than n
while (s <= 100) {
  // s = A + oddsum(n)
  if (n % 2 > 0) {

    s += n;

  } else {

  }
  // s = A + oddsum(n+1)  –  common for both branches
  n = n + 1;
  // s = A + oddsum(n)
}
```

The assignment rule for **`n = n + 1`** restores the invariant.

Exercise 2: Solution


```
let n: number = 0;
let s: number = A;
// s = A + oddsum(n)  -  sum of odd numbers less than n
while (s <= 100) {
  // s = A + oddsum(n) && s <= 100
  if (n % 2 > 0) {

    s += n;

  } else {

  }

  n = n + 1;
  // s = A + oddsum(n)
}
// n = 20 & A + oddsum(n-1) <= 100 && A + oddsum(n) > 100
```

To find **A**, we impose:
s <= 100 for **n**=19 (enter loop)
s > 100 for **n**=20 (exit loop)

We have **oddsum(20)** = 1+3+..+19
= 10*(1+19)/2 = 100

A + (100-19) <= 100 && **A**+100 > 100

0 < **A** <= 19

The largest value is 19.